

Auditor Bancario VCT — Informe público: código fuente explicado

Versión: 2.12.11 · **Público** — apto para GitHub, Patreon y NotebookLM

Autor: Angel del Valle Calero (calero89) · © 2026 · Vanilla Center Trust [VCT]

Sin datos de infraestructura, tokens ni IDs de servidor

1. ¿Qué es este proyecto en código?

Bot **Python 3** + **discord.py** con arquitectura modular:

```
Auditor_Bancario_bot.py      # Entrada
├── vct_auditor/              # Paquete principal
│   ├── config.py             # Reglas de juego (constantes)
│   ├── bot_core.py           # Bot + eventos Discord
│   ├── storage.py            # Persistencia JSON
│   ├── commands/             # ~89 slash commands
│   └── [módulos de dominio]   # economía, FS22, crimen, tiendas...
```

2. Arranque en 4 pasos

```
# 1. Cargar token desde entorno (nunca hardcodeado)
DISCORD_BOT_TOKEN = _cargar_variable_env("DISCORD_BOT_TOKEN")

# 2. Crear bot con intents
intents = discord.Intents.default()
intents.members = True
intents.message_content = True

# 3. Al conectar: cargar JSON, registrar comandos, sync slash
async def setup_hook(self):
    self.cargar_datos()
    register_commands(self.tree)
    await self.tree.sync(guild=guild)

# 4. Ejecutar
bot.run(DISCORD_BOT_TOKEN)
```

3. Constantes de economía (config.py)

Todas las reglas de juego viven en un solo archivo:

```
IMPUESTO_TRANSACCION = 0.05      # 5 % transferencias legales
IMPUESTO_SEMANAL = 0.20          # Lunes sobre cartera
```

```

ATRACO_EXITO_CHANCE = 0.35      # 35 % atraco en equipo
CONTRABANDO_INSPECCION_CHANCE = 0.30
TRABAJOS_FS22 = {
    "cosecha": "Cosecha",
    "hilarar": "Hilarar",
    # ... 36 tipos
}

```

Ventaja de diseño: cambiar economía sin tocar lógica de comandos.

4. Persistencia segura (storage.py)

Versión **didáctica** (autocontenida). En producción se añaden carpeta destino, backup .ultimo_ok y reintentos ante PermissionError. El mismo fragmento vive en ejemplos/06_persistencia_json_atomica.py.

```

import json
import os
import time

def guardar_json_atomico(ruta: str, datos) -> None:
    """Escribe `datos` en `ruta` vía fichero temporal + replace + fsync."""
    temporal = f"{ruta}.{os.getpid()}.{time.time_ns()}.tmp"
    try:
        with open(temporal, "w", encoding="utf-8") as f:
            json.dump(datos, f, indent=4, ensure_ascii=False)
            f.flush()
            os.fsync(f.fileno())
        os.replace(temporal, ruta) # Atómico en POSIX/Windows
    finally:
        if os.path.exists(temporal):
            try:
                os.remove(temporal)
            except OSError:
                pass

```

Cada cuenta de jugador es un objeto en banco_vct.json indexado por ID Discord.

5. Lock anti double-spend (banco_sync.py)

El fsync del JSON no debe bloquear el event loop ni otros slash (límite Discord 3 s). Tras mutar en memoria se hace snapshot (también si el comando falla) y se persiste en un hilo; los guardados se serializan para que una foto antigua no sobrescriba una más reciente:

```

import asyncio
import copy
from contextlib import asynccontextmanager

@asynccontextmanager
async def transaccion_banco(*, persistir: bool = True):
    async with bot._banco_lock:
        snapshot = None
        try:
            yield bot
        finally:
            if persistir:
                snapshot = copy.deepcopy(bot.banco)
    if snapshot is not None:
        async with bot._banco_persist_lock:
            await asyncio.to_thread(guardar_json_atómico, ruta_banco(), snapshot)

```

Todos los slash commands se envuelven al registrarse:

```

# commands/__init__.py
def register_commands(tree):
    bank.setup(tree)
    crime.setup(tree)
    # ...
    envolver_arbol_comandos(tree)  # ← último paso

```

6. Comando slash típico

```

@tree.command(name="depositar", description="Mover dinero de cartera a caja fuerte")
async def depositar(interaction: discord.Interaction, monto: int):
    bot = get_bot()
    bot.check_user(interaction.user.id)  # Crea cuenta si no existe
    datos = bot.banco[str(interaction.user.id)]
    if monto <= 0:
        await interaction.response.send_message("❌ El monto debe ser positivo.", ephemeral=True)
        return
    if datos["balance"] < monto:
        await interaction.response.send_message("❌ Saldo insuficiente.", ephemeral=True)
        return
    datos["balance"] -= monto
    datos["vault"] += monto
    bot.guardar_datos()
    await interaction.response.send_message(f"✅ Guardados {monto} €", ephemeral=True)

```

(En producción el guardar_datos ocurre dentro del lock automático.)

7. Autocompletado FS22 (límite Discord 25)

Discord no admite más de 25 choices estáticas. Solución:

```
def choices_autocomplete_trabajos_fs22(current: str) -> list[tuple[str, str]]:
    busqueda = normalizar(current)
    candidatos = filtrar(TRABAJOFS22, busqueda)
    return candidatos[:25] # Tope API Discord

@tree.command(name="registrar_contrato_fs22", ...)
@app_commands.autocomplete(tipo=_autocomplete_tipo_fs22) # @tree.command PRIMERO
async def registrar_contrato_fs22(interaction, tipo: str):
    ...
```

8. Guard de arresto en tiendas

```
async def bloquear_si_arrestado(interaction, banco) -> bool:
    uid = str(interaction.user.id)
    if uid not in banco or not esta_arrestado(banco[uid]):
        return False
    await interaction.response.send_message(MENSAJE_BLOQUEO_ARRESTO, ephemeral=True)
    return True
```

Se llama al inicio de cada botón de catálogo persistente.

9. Persecución y prisión (flujo)

```
/robar (éxito) → NRD + registro persecución 24h
↓
/perseguir (víctima) → minijuego
↓ captura
arrestar_jugador() → incauta NRD/inventario, rol Recluso
↓
publicar_expediente_penitenciario() → canal #prisión
↓
/pagar_fianza_vct o /trabajos_comunitarios x5
↓
publicar_limpieza_expediente() → libertad + antecedentes borrados
```

10. Contrabando FS22 (v2.9.6)

```
CONTRABANDO_FS22_PRODUCTOS = {
    "trigo_negro": ("Trigo fuera de contrato", "agricultura", 8000, 18000, 4, 7),
    # nombre, sector, nrd_min, nrd_max, rep_min, rep_max
```

```

}

if random.random() < 0.30:
    # Inspección: multa legal + antecedente
else:
    ganancia = random.randint(nrd_min, nrd_max)
    datos["dinero_negro"] += ganancia
    modificar_reputacion(datos, criminal=rep_ganada)

```

11. Tiendas persistentes (shop_views.py)

```

class TiendaLegalView(ui.View):
    def __init__(self, banco):
        super().__init__(timeout=None) # Persistente
        self.banco = banco

    @ui.button(label="Comprar", custom_id="vct_tienda_legal_comprar")
    async def comprar(self, interaction, button):
        if await bloquear_si_arrestado(interaction, self.banco):
            return
        # lógica compra...

```

custom_id fijo permite que el bot recuerde botones tras reinicio.

12. Monetización dual

Canal	Módulo	Resultado
Ko-fi webhook	kofi.py	Rol Ciudadano / Socio Preferente
Discord SKU	monetization.py	Rol Socio VCT + /verificar_socio_vct

```

SKU_SOCIO_VCT = int(_cargar_variable_env("DISCORD_SKU_SOCIO_VCT_ID", "0") or "0")

async for ent in bot.entitlements(exclude_ended=True):
    if ent.sku_id == SKU_SOCIO_VCT:
        await otorgar_rol_socio_vct(miembro)

```

13. Más ejemplos en el repositorio

Carpeta ejemplos/:

Archivo	Tema
01_economia_constantes.py	Tasas y FS22
02_carga_variables_entorno.py	kofi.env
03_guard_arresto_tiendas.py	Bloqueo reclusos
04_fs22_autocomplete.py	Autocompletado
05_lock_transacciones_banco.py	Lock async
06_persistencia_json_atomica.py	JSON atómico (tmp + fsync)

14. Stack y dependencias principales

- discord.py 2.x — API Discord
- aiohttp — webhook Ko-fi
- JSON local — sin base de datos externa
- Servicio 24/7 en Linux

© 2026 Angel del Valle Calero · Vanilla Center Trust [VCT] · Informe público de código
· v2.9.7